

DATA PIPELINE DESIGN

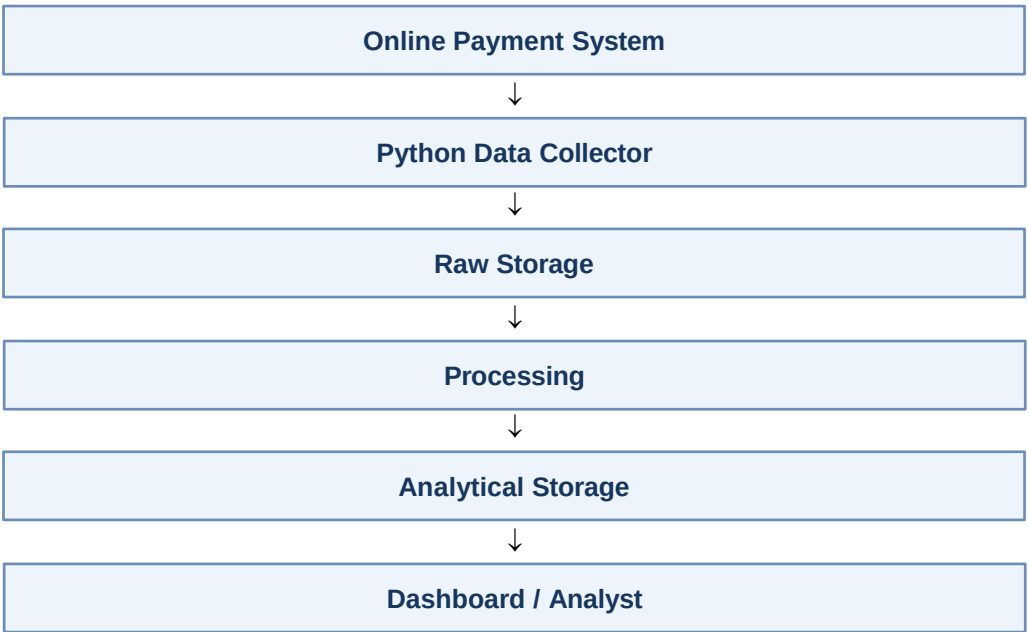
Online Payment System — A Simple End-to-End Payment Data Pipeline

Selected Data Source	Online Payment System
Pipeline Type	Transaction / Periodic Data Collection
Primary Use Case	Payment transaction analysis and dashboard reporting

Objective

Design a simple data pipeline that explains where payment transaction data comes from, how it enters the pipeline, how it is processed, where it is stored, and who uses it.

Pipeline at a Glance



1. Selected Source

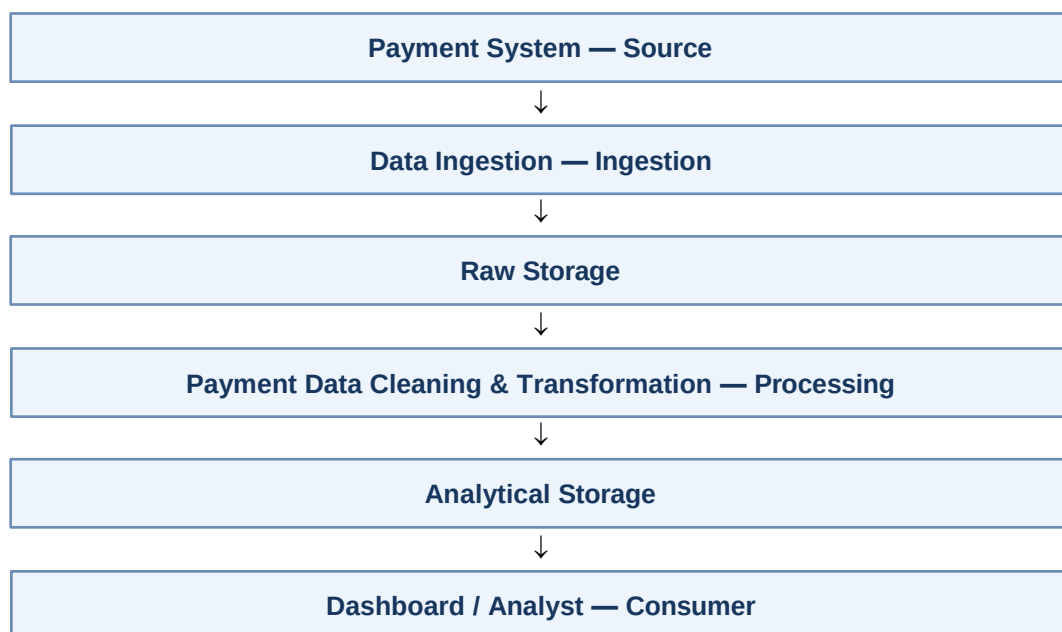
The selected source is an Online Payment System. It generates transaction information when customers make payments for products or services.

Example data fields:

- Transaction ID
- Customer ID
- Amount
- Payment Method
- Transaction Status
- Date and Time

2. Diagram 1 — Logical Architecture

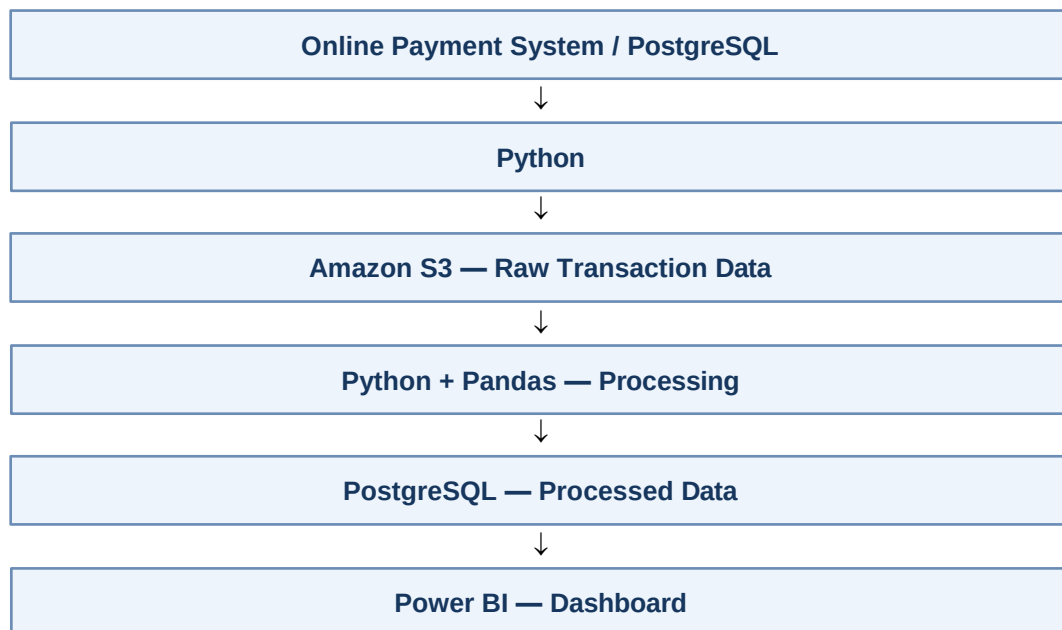
The logical architecture focuses on what each stage does rather than on specific technology products.



Logical flow: Source → Ingestion → Raw Storage → Processing → Analytical Storage → Consumer.

3. Diagram 2 — Technology Mapping

The logical components are mapped to practical technologies. These choices are intentionally simple and are not intended to be the only possible solution.



Technology flow: Payment System / PostgreSQL → Python → Amazon S3 → Pandas → PostgreSQL → Power BI.

4. Component Explanation

Component	Why it exists
Online Payment System / PostgreSQL	Provides the source transaction records generated by customer payments.
Python	Connects to the source and collects payment transaction data for the pipeline.
Amazon S3	Stores the original raw transaction data so it is preserved before transformation.
Pandas	Cleans and transforms payment records into a consistent structured format.
PostgreSQL	Stores processed payment records in structured tables for efficient querying.
Power BI	Uses processed payment data to create dashboards and reports for users and analysts.

5. Processing Stage

The processing stage converts raw payment transaction records into clean and useful analytical data.



For example, a payment amount received as text can be converted to a numeric value. Duplicate transaction records can also be removed before the final data is stored.

Short Architecture Explanation

The pipeline starts with the Online Payment System, which is the source of transaction data. A Python program collects the payment records and sends them into the data pipeline. The collected data is first preserved as raw data in Amazon S3 so the original records are available for reprocessing.

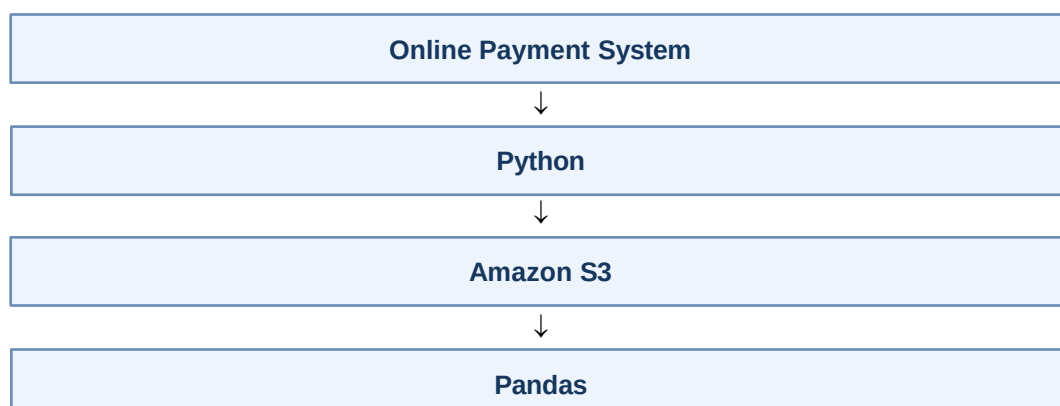
The raw transaction data is then processed using Pandas. During processing, invalid records, duplicates, missing values, and inconsistent formats are handled. The cleaned data is stored in PostgreSQL, where it can be queried efficiently for reporting and analysis.

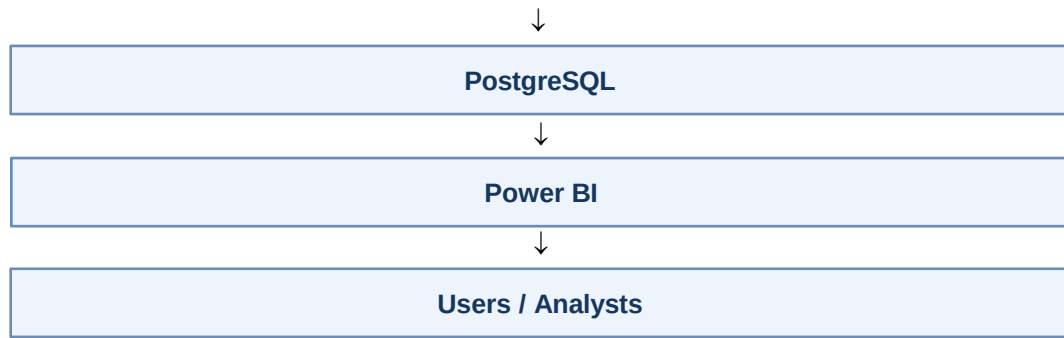
Finally, Power BI connects to the processed data and presents dashboards showing information such as transaction volume, successful and failed payments, payment methods, and transaction trends.

Why does data move from one component to the next?

Each stage has a clear responsibility: the payment system produces transaction data, Python collects it, S3 preserves the raw data, Pandas prepares it for analysis, PostgreSQL stores structured records, and Power BI presents the results to users. This separation keeps the pipeline simple and understandable.

6. Final Architecture Summary





Final pipeline: Online Payment System → Python → Amazon S3 → Pandas → PostgreSQL → Power BI → Users.